

# **CPE 324-03: Advanced Digital Logic Design Laboratory**

## **Lab 7**

### **Digital to Analog Conversion using Pulse Density Modulation**

**Submitted by:** Hannah Kelley

**Date of Experiment:** March 17, 19, and 24, 2026

**Report Deadline:** March 31, 2026

**Demonstration Deadline:** March 24, 2026

**Lab Partner:** Margaret Roberts

# 1 Introduction

In this laboratory, digital hardware was used to explore the generation of analog signals from discrete-time systems. Specifically, the DE2-115 FPGA and its I/O pins were utilized to implement a pulse density modulation (PDM) signal processing algorithm, demonstrating how digital systems can approximate continuous signals. In Part A, four static waveforms were generated and analyzed using an oscilloscope and multimeter to evaluate their analog characteristics. In Part B, the design was extended to enable playback of a prerecorded, sampled audio signal through an attached speaker. Finally, in Part C, the system was further modified to produce an amplitude-modulated (AM) radio frequency signal based on the same audio input, illustrating a practical application of modulation techniques.

## 2 Experiment Description

### 2.1 Part A Description – Analog Waveform Generator

#### Evaluation/Simplification

In Part A of this laboratory, the given template was used to analyze three existing waveforms generation methods using an oscilloscope and digital multimeter. The existing waveform types analyzed were square, triangle, and sine waves. After analyzing these three waves, a new sawtooth waveform segment was added.

After analyzing the waveforms, the design was modified to combine the Clock\_en and Waveform\_Gen modules into a single module named Waveform\_Gen2. Figure 1 shows the original design and the simplified design side-by-side.

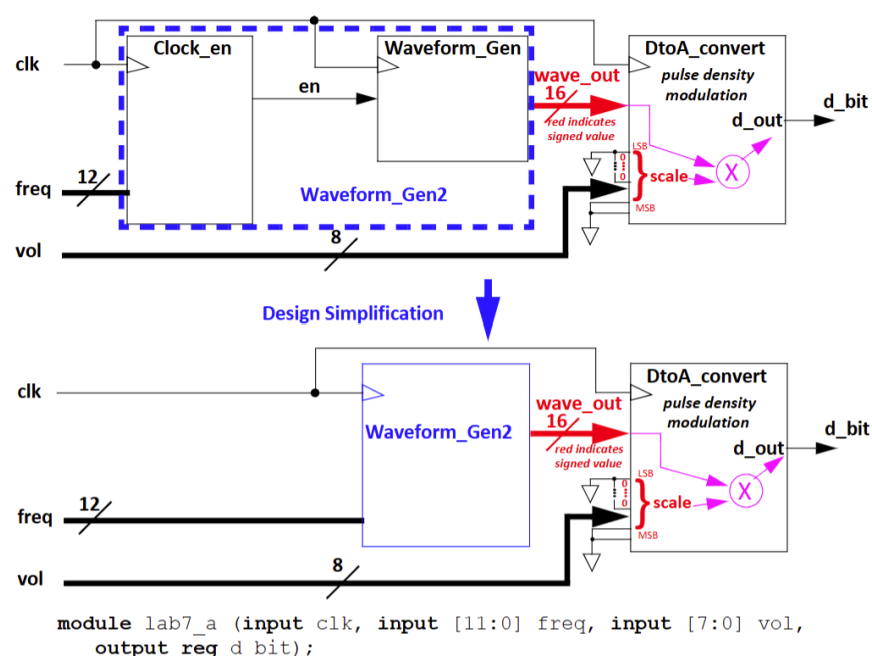


Figure 1: Design Simplification Diagram

## 2.2 Part B Description

Part B built off of the concepts and revised code in Part A to construct an audio generator capable of turning pre-recorded and sampled (8 kHz) audio from a text file into audio output from the speaker. To do this, the Waveform\_Gen2 module was modified into the Audio\_Out module by using the \$readmemh directive to initialize a 2D array of the file contents. The module can handle 8- and 16-bit data in hexadecimal form from text files. The top-level structural model of this design can be found in Figure 2.

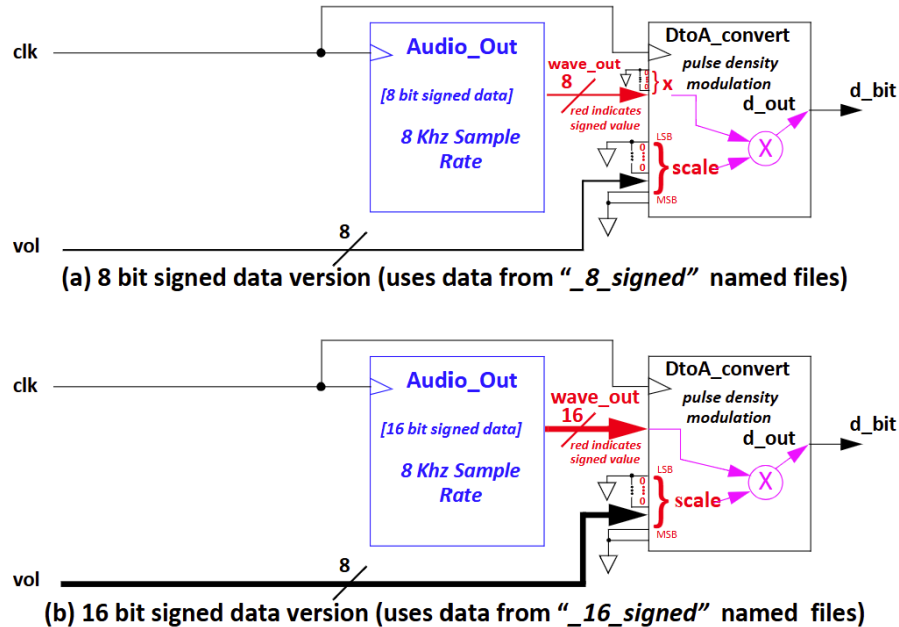


Figure 2: Audio Generator Diagram

## 2.3 Part C Description

The final portion of this lab continued to build off of the previous parts to play a sampled 1 MHz RF sine wave to be amplitude modulated by a 8 kHz sampled audio wave. The combination of these two waveforms can be found in Figure 3.

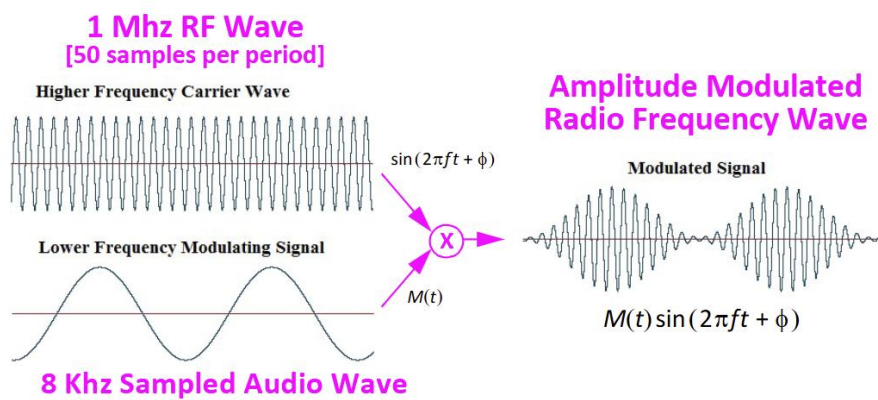
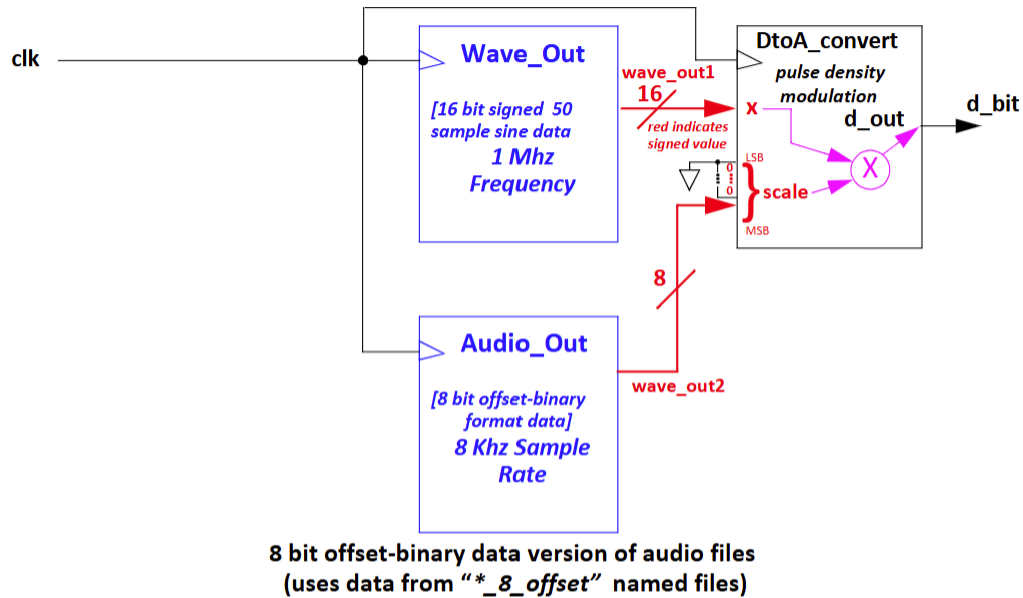


Figure 3: Amplitude Modulation Diagram

The final design allows the audio to be wirelessly broadcast over short distances and received by a standard AM receiver. The Audio\_Out module from Part B was modified to use sampled 8 kHz offset data to drive wave\_out2 and a sine lookup table to drive wave\_out. The final design can process 8- and 16-bit data. The final design's high-level model can be found in Figure 4.



```
module lab7_c (input clk, output reg d_bit);
```

**Figure 4: AM Generator Design Diagram**

## 3 Results

### 3.1 Part A Results

Figures 5, 6, and 7 show the waveform characteristics for each of the three waveforms generated using code from the lab template. Table 1 shows the sawtooth waveform generator code segment added to the template. Figure 8 shows the waveform characteristics for the sawtooth waveform.

## 1 KHz Waveform -- Square Wave

### Oscilloscope Settings

Volts per division 1.68V

Time per division 500  $\mu$ s

### Measured (oscilloscope)

Peak-to-Peak Voltage 3.5V

Frequency 1.02 kHz

### Measured (Digital Multimeter)

AC Voltage 1.668V

DC Voltage 1.514V

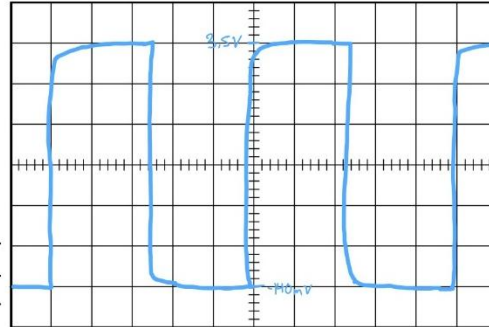
### Speaker Observations:

constant tone

### Waveform Observations:

rounded corners

### Waveform Sketch



if possible adjust time mode so that two complete periods of the waveform are displayed

### Other Observations:

Figure 5: Square Wave Characteristics

## 1 KHz Waveform -- Triangle Wave

### Oscilloscope Settings

Volts per division 1.67V

Time per division 200  $\mu$ s

### Measured (oscilloscope)

Peak-to-Peak Voltage 3.55V

Frequency 1.02 kHz

### Measured (Digital Multimeter)

AC Voltage 6.967V

DC Voltage 1.666V

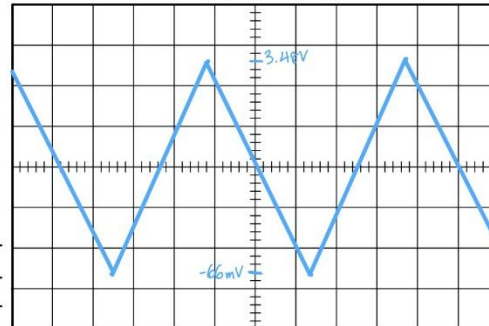
### Speaker Observations:

higher frequency constant tone  
very grating to the ears

### Waveform Observations:

triangle waveform

### Waveform Sketch



if possible adjust time mode so that two complete periods of the waveform are displayed

### Other Observations:

Figure 6: Triangle Wave Characteristics

## 1 Khz Waveform -- Sine Wave

Oscilloscope Settings  
 Volts per division 1.7V  
 Time per division 500 $\mu$ s  
 Measured (oscilloscope)  
 Peak-to-Peak Voltage 3.67V  
 Frequency 1.02kHz  
 Measured (Digital Multimeter)  
 AC Voltage 1.182V  
 DC Voltage 1.628V

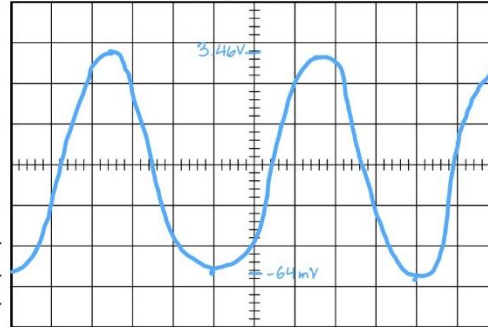
Speaker Observations:

constant tone lower than that of the  
square wave

Waveform Observations:

sinusoid

Waveform Sketch



if possible adjust time mode so that two complete periods of the waveform are displayed

Other Observations:

### Figure 7: Sine Wave Characteristics

```
reg [5:0] i;
reg signed [1:-14] delta, val;
delta = `ONE_16 * 2 / 50;
val = -`ONE_16;
for (i = 6'd0; i < 6'd50; i = i + 6'd1)
begin
  out_table[i] = val;
  val = val + delta;
end
```

### Table 1: Sawtooth Waveform Code Segment

## 1 Khz Waveform -- Sawtooth Wave

Oscilloscope Settings  
 Volts per division 1.67V  
 Time per division 200 $\mu$ s  
 Measured (oscilloscope)  
 Peak-to-Peak Voltage 3.43V  
 Frequency 1.02kHz  
 Measured (Digital Multimeter)  
 AC Voltage 0.965V  
 DC Voltage 1.636V

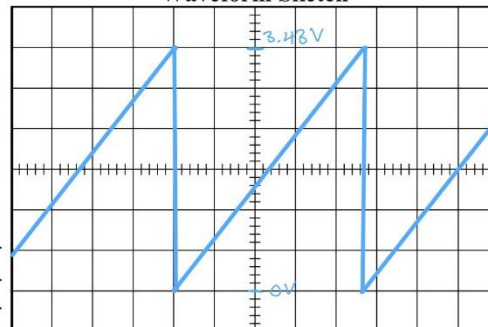
Speaker Observations:

constant high pitch tone

Waveform Observations:

very noisy signal, reduced noise for  
the sketch

Waveform Sketch



if possible adjust time mode so that two complete periods of the waveform are displayed

Other Observations:

### Figure 8: Sawtooth Wave Characteristics

During the waveform analysis, the following questions were considered:

**1. What is the effect of varying the switches SW17 through SW10?**

At a logic low these switches reduce the peak-to-peak voltage and the volume from the speaker. The higher number the switch the greater the change in voltage and volume.

**2. What is the effect of varying the switches SW9 through SW0 and KEY3/KEY2?**

These switches and keys reduce the frequency of the signal when set to a logic low. Like with switches 10-17, the higher the switch or key number the greater the change.

**3. What is the slowest frequency signal that can be produced?**

The lowest frequency signal produced in this lab is a 0 Hz signal. This can be accomplished by setting all of the frequency switches to a logic low.

**4. What is the highest frequency signal?**

The highest frequency signal produced in this lab was 4.08 kHz. This can be accomplished by setting all of the frequency switches to a logic high and holding keys 2 and 3 down.

**5. What is the frequency range that produces audio frequency that can be heard through the speaker?**

A frequency of 1 Hz to 4.08 kHz can be heard through the speaker.

**6. When the digital multimeter is in the AC mode, does it measure RMS voltage or something else? What about when it is in DC mode? If it does not measure RMS voltage then what does it appear to measure?**

The digital multimeter measures RMS voltage in AC mode. In DC mode, the multimeter measure the average voltage (aka steady-state voltage) of the DC signal.

**7. How close was the measured frequency to 1 kHz? Explain what could be causing any differences between measured and the frequency that the design is supposed to produce?**

The measured frequency of 1.02 kHz is extremely close to the desired 1 kHz frequency. Since the frequency is expressed in binary using switches 10-17, this is as close to the desired frequency as is physically possible. The frequency did fluctuate occasionally, most likely due to interference or imperfections in the equipment, especially the resistor.

Table 2 shows the Waveform\_Gen2 code segment, a combination of the Clock\_en and Waveform\_Gen modules into a single module.

```
always @(posedge clk) begin : Waveform_Gen2
    // Declaring local variables in named section 'clock_en'
    reg [19:0] count;
    reg [19:0] base_count;
    reg [5:0] index;

    if (freq) begin
        base_count = 20'd1000000 / freq;
```

```

    if (count) begin
        count = count - 20'd1;
        // en = 1'b0;
    end else begin
        // en = 1'b1;
        if (index >= 6'd49)
            index = 6'd0;
        else
            index = index + 6'd1;

        wave_out = out_table[index];
        count = base_count;
    end
end
end
end

```

**Table 2: Waveform\_Gen2 Code Segment**

### 3.2 Part B Results

Table 3 shows the Audio\_Out module. No physical results exist for part B. The audio file used in Part B could be heard playing from the speaker.

```

begin : Audio_Out
    reg [12:0] count;
    reg [12:0] base_count;
    reg [16:0] index;

    base_count = 13'd6250;

    if (count) begin
        count = count - 13'd1;
    end else begin
        if (index >= 17'd4873)
            index = 17'd0;
        else
            index = index + 17'd1;

        wave_out2 = audio[index];
        count = base_count;
    end
end
end

```

**Table 3: Complete Audio\_Out Module**

After successfully implementing Part B, the following question was considered:

- **Given the size of the Altera Cyclone IV E EP4CE115F29C7 FPGA assuming that all the audio memory is stored in embedded RAM and not created from other FPGA**

**logic elements, what is the maximum length (un-compressed) that can be supported for both the 8 bit case and the 16 bit case without exceeding the available resources?**

The Altera Cyclone IV E FPGA provides a total on-chip memory capacity of 3,981,312 bits. To estimate how long an audio recording can be stored, we divide this memory size by both the bit depth (bits per sample) and the sampling frequency.

For 8-bit audio sampled at 8 kHz:  $maximum\ duration \approx \frac{3,981,312}{8 \cdot 8000} \approx 62\ sec$

For 16-bit audio at the same 8 kHz rate:  $maximum\ duration \approx \frac{3,981,312}{16 \cdot 8000} \approx 31\ sec$

These figures are approximate and assume ideal usage of all available memory. In practice, the actual recording time may be slightly shorter due to implementation overhead and other resource demands within the design.

### 3.3 Part C Results

After successfully implementing Part C, the AM radio frequency wave could be heard using the handheld radio receiver. The Audio\_Out module used in Part C can be found in Table 4.

```
begin : Audio_Out
    reg [12:0] count;
    reg [12:0] base_count;
    reg [16:0] index;

    base_count = 13'd6250;

    if (count) begin
        count = count - 13'd1;
    end else begin
        if (index >= 17'd4873)
            index = 17'd0;
        else
            index = index + 17'd1;

        wave_out = audio[index];
        count = base_count;
    end
end
```

**Table 4: Complete Audio\_Out Module**

### 3.4 Post-Lab Questions

After concluding the lab, the following questions were considered:

- 1. Why was signed arithmetic used in the PDM task? How might signed arithmetic help in terms of modeling the problem?**

Signed arithmetic was used in the PDM task to represent both positive and negative signal values, which is essential for accurately modeling waveforms that fluctuate above

and below zero. This allows the digital system to properly track signal deviations and produce symmetric analog outputs. Using signed arithmetic simplifies the design and ensures that the Pulse Density Modulation correctly reflects the full amplitude range of the input waveform.

**2. What are the advantages and disadvantages of floating point arithmetic as opposed to integer or fixed-point arithmetic in digital hardware? What favored the use of fixed point in this design?**

Floating-point arithmetic offers a wide dynamic range and high precision, making it ideal for complex calculations. However, it is slower, requires more hardware resources, and consumes more power compared to integer or fixed-point arithmetic. Fixed-point arithmetic, in contrast, is faster, simpler, and more resource-efficient, which makes it better suited for FPGA designs like this lab where real-time signal generation is required. The design favored fixed-point arithmetic to maximize speed and minimize FPGA resource usage while maintaining sufficient accuracy for waveform and audio output.

**3. Why are lookup tables often used to generate sine waves? Why not just compute on the fly using a numeric methods to generate the sine function?**

Lookup tables (LUTs) are used to generate sine waves because they provide fast, deterministic, and resource-efficient output. By storing precomputed values, the system can produce sine wave samples in a single clock cycle, avoiding the multiple calculations required by numerical methods like Taylor series expansions. LUTs also reduce the use of complex arithmetic hardware on FPGAs, simplify design, and minimize rounding or approximation errors. Overall, they trade a small amount of memory for high-speed, accurate, and simple waveform generation.

## 4 Conclusion

This lab focused on applying digital hardware principles to the synthesis and manipulation of analog signals. An FPGA-based system was developed to generate a range of waveform types, support playback of sampled audio, and implement an amplitude modulation approach for radio frequency transmission. Key techniques, including pulse density modulation and fixed-point arithmetic, enabled precise and resource-efficient handling of audio data within the limitations of the Altera Cyclone IV E FPGA. Sine wave generation was achieved through lookup tables, allowing continuous real-time operation without excessive computational cost. Collectively, the results of this experiment demonstrate the effectiveness of FPGA-based design in implementing practical signal processing functions and producing complex analog behaviors from digital systems.

During the experiment, some challenges arose while using the oscilloscope. The auto-adjust feature did not reliably display the signals, requiring manual adjustment of the timebase, voltage scale, and trigger settings to properly observe the waveforms. Although this added extra steps to

the measurement process, it provided a better understanding of how the oscilloscope operates and ensured accurate visualization of the generated signals.

## Appendix A – Final lab6.v File

```
// lab6 Template
// Waveform Generation using Pulse Density Modulation

`timescale 1ns/1ps

module Lab7_Part_a (
    input clk,
    input [11:0] freq,
    input [7:0] vol,
    output reg d_bit
);

    // 16-bit signed integer 50-point lookup table for wave function
    reg signed [1:-14] audio[0:4873];
    reg signed [1:-14] out_table[49:0];
    reg signed [1:-14] wave_out1;
    reg signed [1:-14] wave_out2;

    initial begin
        $readmemh("audio_01_16_signed.txt", audio);
        // 16-bit signed fixed-point encoding of 1.0
        // 1 bit mantissa and 14-bit fraction
        `define ONE_16 {1'b0,1'b1,14'b0}

        /* en
        // load out_table with 50 point 16 bit integer
        // square wave lookup table information
        // using signed notation
        // Note: lookup table would not be really be needed
        // for such a simple waveform but is used for the
        // sake of generality.
        reg [5:0] i;
        for (i=6'd0; i<6'd25;i=i+6'd1) out_table[i] = `ONE_16;
        for (i=6'd25;i<6'd50;i=i+6'd1) out_table[i] = -`ONE_16;
        */

        /*
        // load out_table with 50 point 16 bit integer
        // triangle wave lookup table information
        // using signed notation
        reg [5:0] i;
```

```

reg signed [1:-14] delta, val;
delta=`ONE_16*2/25;
val=`ONE_16;
for (i=6'd0; i<6'd25;i=i+6'd1) begin
    out_table[i]= val;
    val = val - delta;
end
for (i=6'd25; i<6'd50;i=i+6'd1) begin
    out_table[i]= val;
    val = val + delta;
end
*/

// load out_table with
// 16 bit integer 50 point sine function
// lookup table
//      Fixed-Point  Aprox Val
out_table[0] = 16'h0000; // 0.000000
out_table[1] = 16'h0805; // 0.125333
out_table[2] = 16'h0fea; // 0.248690
out_table[3] = 16'h178f; // 0.368125
out_table[4] = 16'h1ed4; // 0.481754
out_table[5] = 16'h259d; // 0.587785
out_table[6] = 16'h2bcf; // 0.684547
out_table[7] = 16'h314f; // 0.770513
out_table[8] = 16'h3609; // 0.844328
out_table[9] = 16'h39e8; // 0.904827
out_table[10] = 16'h3cdd; // 0.951057
out_table[11] = 16'h3edd; // 0.982287
out_table[12] = 16'h3fdf; // 0.998027
out_table[13] = 16'h3fdf; // 0.998027
out_table[14] = 16'h3edd; // 0.982287
out_table[15] = 16'h3cdd; // 0.951057
out_table[16] = 16'h39e8; // 0.904827
out_table[17] = 16'h3609; // 0.844328
out_table[18] = 16'h314f; // 0.770513
out_table[19] = 16'h2bcf; // 0.684547
out_table[20] = 16'h259d; // 0.587785
out_table[21] = 16'h1ed4; // 0.481754
out_table[22] = 16'h178f; // 0.368125
out_table[23] = 16'h0fea; // 0.248690
out_table[24] = 16'h0805; // 0.125333
out_table[25] = 16'h0000; // 0.000000
out_table[26] = 16'hf7fb; // -0.125333
out_table[27] = 16'hf016; // -0.248690
out_table[28] = 16'he871; // -0.368125

```

```

out_table[29] = 16'he12c; // -0.481754
out_table[30] = 16'hda63; // -0.587785
out_table[31] = 16'hd431; // -0.684547
out_table[32] = 16'hceb1; // -0.770513
out_table[33] = 16'hc9f7; // -0.844328
out_table[34] = 16'hc618; // -0.904827
out_table[35] = 16'hc323; // -0.951057
out_table[36] = 16'hc123; // -0.982287
out_table[37] = 16'hc021; // -0.998027
out_table[38] = 16'hc021; // -0.998027
out_table[39] = 16'hc123; // -0.982287
out_table[40] = 16'hc323; // -0.951057
out_table[41] = 16'hc618; // -0.904827
out_table[42] = 16'hc9f7; // -0.844328
out_table[43] = 16'hceb1; // -0.770513
out_table[44] = 16'hd431; // -0.684547
out_table[45] = 16'hda63; // -0.587785
out_table[46] = 16'he12c; // -0.481754
out_table[47] = 16'he871; // -0.368125
out_table[48] = 16'hf016; // -0.248690
out_table[49] = 16'hf7fb; // -0.125333

// Sawtooth Wave
/*
Your turn!!!
Add your own modelling Code here
\_ (ツ) \_
*/
//
// reg [5:0] i;
// reg signed [1:-14] delta, val;
// delta=`ONE_16*2 /50;
// val=-`ONE_16;
// for (i=6'd0; i<6'd50;i=i+6'd1)
//   begin
//     out_table[i]= val;
//     val = val + delta;
//   end
// end
end

always @(posedge clk) begin : Audio_Out
  reg [12:0] count;
  reg [12:0] base_count;
  reg [16:0] index;

```

```

base_count = 13'd6250;
if (count) begin
    count = count - 13'd1;
end else begin
    if (index >= 17'd4873)
        index = 17'd0;
    else
        index = index + 17'd1;

    wave_out2 = audio[index];
    count = base_count;
end
end

always @(posedge clk) begin : Wave_Out
    reg [5:0] index;
    if (index >= 6'd49)
        index = 6'd0;
    else
        index = index + 6'd1;

    wave_out1 = out_table[index];
end

always @(posedge clk) begin : DtoA_convert
    reg signed [3:-28] ge; // error term
    pdm(d_bit, wave_out1, wave_out2, ge);
end

// Pulse Density Modulation Task
task pdm(
    output d_out,
    input signed [1:-14] x,
    scale,
    inout signed [3:-28] ge
);
    `define ONE_32 {1'b0,3'b001,28'b0} // 32-bit fixed-point 1.0
    reg signed [3:-28] x_total;

    begin
        x_total = x * scale; // signal multiplication

        if (x_total >= ge) begin
            d_out = 1;
            ge = ge + (`ONE_32 - x_total);
        end else begin

```

```
        d_out = 0;
        ge = ge - (`ONE_32 + x_total);
    end
end
endtask

endmodule
```